

车辆维修管理系统 - 核心功能SQL语句说明

生成日期：2025-01-12
数据库：MySQL 8.0+
ORM框架：Sequelize (Node.js)

1. 概述

本文档详细说明车辆维修管理系统中核心业务功能对应的SQL语句实现。系统使用Sequelize ORM框架，但为了更好地理解数据库操作逻辑，这里提供了等效的原生SQL语句。

2. 用户管理相关SQL

2.1 用户注册

功能描述：新用户注册，根据角色自动创建相应档案

```
-- 创建新用户
INSERT INTO users (role, name, password_hash, created_at)
VALUES ('customer', '张三', '$2b$10$...', NOW());

-- 如果是技师角色，自动创建技师档案
INSERT INTO mechanic_profiles (mechanic_id, hire_date)
VALUES (LAST_INSERT_ID(), CURDATE());
```

对应Sequelize代码：

```
// User.createUser() 方法
const user = await User.create({
  role: 'customer',
  name: '张三',
  password_hash: hashedPassword,
  created_at: new Date()
});
```

2.2 用户登录验证

功能描述：验证用户登录凭据

```
-- 根据用户名查找用户
SELECT user_id, role, name, password_hash, created_at
FROM users
WHERE name = '张三';
```

2.3 获取技师列表

功能描述：按专业工种查找可用技师

```
-- 查找指定工种的技师
SELECT
    u.user_id,
    u.name,
    mp.trade,
    mp.hourly_rate,
    mp.cert_no,
    COUNT(wom.order_id) as active_orders
FROM users u
INNER JOIN mechanic_profiles mp ON u.user_id = mp.mechanic_id
LEFT JOIN work_order_mechanics wom ON mp.mechanic_id = wom.mechanic_id
    AND wom.status IN ('not_started', 'in_progress')
WHERE u.role = 'mechanic'
    AND mp.trade = 'engine'
GROUP BY u.user_id, u.name, mp.trade, mp.hourly_rate, mp.cert_no
HAVING active_orders < 5 -- 限制技师同时处理的工单数量
ORDER BY active_orders ASC, mp.hourly_rate ASC;
```

3. 工单管理相关SQL

3.1 创建工单

功能描述：客户创建新的维修工单

```
-- 插入新工单
INSERT INTO work_orders (
    vehicle_id,
    customer_id,
    status,
    description,
    trade,
    created_at
) VALUES (
    1001,
    2001,
    'pending',
    '发动机异响，需要检查',
    'engine',
    NOW()
);

-- 同时创建支付记录
INSERT INTO payments (order_id, labor_fee, material_fee, total_fee)
VALUES (LAST_INSERT_ID(), 0, 0, 0);
```

3.2 工单分配

功能描述：管理员将工单分配给技师

```
-- 分配技师到工单
INSERT INTO work_order_mechanics (order_id, mechanic_id, status, note)
VALUES (10001, 3001, 'not_started', '已分配给李师傅');

-- 更新工单状态
UPDATE work_orders
SET status = 'assigned'
WHERE order_id = 10001;
```

3.3 工单进度查询

功能描述：查询工单详细信息及进度

```
-- 查询工单详细信息
SELECT
    wo.order_id,
    wo.description,
    wo.status as order_status,
    wo.trade,
    wo.created_at,
    wo.finished_at,
    -- 客户信息
    customer.name as customer_name,
    -- 车辆信息
    v.plate_no,
    v.model,
    v.year,
    -- 分配的技师信息
    GROUP_CONCAT(
        CONCAT(mechanic.name, '(', wom.status, ':', wom.hours_worked, 'h)')
        SEPARATOR ', '
    ) as assigned_mechanics,
    -- 支付信息
    p.labor_fee,
    p.material_fee,
    p.total_fee,
    p.paid_at
FROM work_orders wo
LEFT JOIN users customer ON wo.customer_id = customer.user_id
LEFT JOIN vehicles v ON wo.vehicle_id = v.vehicle_id
LEFT JOIN work_order_mechanics wom ON wo.order_id = wom.order_id
LEFT JOIN mechanic_profiles mp ON wom.mechanic_id = mp.mechanic_id
LEFT JOIN users mechanic ON mp.mechanic_id = mechanic.user_id
LEFT JOIN payments p ON wo.order_id = p.order_id
```

```
WHERE wo.order_id = 10001
GROUP BY wo.order_id;
```

3.4 技师工单列表

功能描述：技师查看自己的工单列表

```
-- 查询技师的工单列表
SELECT
    wo.order_id,
    wo.description,
    wo.status as order_status,
    wo.created_at,
    wom.status as work_status,
    wom.hours_worked,
    wom.note,
    v.plate_no,
    v.model,
    customer.name as customer_name
FROM work_order_mechanics wom
INNER JOIN work_orders wo ON wom.order_id = wo.order_id
INNER JOIN vehicles v ON wo.vehicle_id = v.vehicle_id
INNER JOIN users customer ON wo.customer_id = customer.user_id
WHERE wom.mechanic_id = 3001
    AND wom.status IN ('not_started', 'in_progress', 'paused')
ORDER BY wo.created_at DESC;
```

4. 库存管理相关SQL

4.1 配件库存查询

功能描述：实时计算配件库存数量

```
-- 计算配件实时库存
SELECT
    p.part_id,
    p.name,
    p.unit,
    p.unit_cost,
    p.qty as system_qty, -- 系统记录的库存
    COALESCE(SUM(it.qty), 0) as calculated_qty, -- 基于交易记录计算的库存
    (p.qty - COALESCE(SUM(it.qty), 0)) as qty_difference -- 差异
FROM parts p
LEFT JOIN inventory_txns it ON p.part_id = it.part_id
GROUP BY p.part_id, p.name, p.unit, p.unit_cost, p.qty
HAVING qty_difference != 0; -- 只显示有差异的记录
```

4.2 库存预警

功能描述：查询库存不足的配件

```
-- 库存预警查询
SELECT
  p.part_id,
  p.name,
  p.unit,
  p.qty as current_stock,
  -- 计算近30天平均消耗量
  COALESCE(
    (SELECT AVG(daily_consumption)
     FROM (
       SELECT DATE(it.created_at) as consumption_date,
              SUM(ABS(it.qty)) as daily_consumption
       FROM inventory_txns it
       WHERE it.part_id = p.part_id
             AND it.type = 'OUT'
             AND it.created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
       GROUP BY DATE(it.created_at)
     ) daily_stats),
    0
  ) as avg_daily_consumption,
  -- 预计可用天数
  CASE
    WHEN COALESCE(
      (SELECT AVG(daily_consumption)
       FROM (
         SELECT DATE(it.created_at) as consumption_date,
                SUM(ABS(it.qty)) as daily_consumption
         FROM inventory_txns it
         WHERE it.part_id = p.part_id
               AND it.type = 'OUT'
               AND it.created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
         GROUP BY DATE(it.created_at)
       ) daily_stats),
      0
    ) > 0
    THEN p.qty / COALESCE(
      (SELECT AVG(daily_consumption)
       FROM (
         SELECT DATE(it.created_at) as consumption_date,
                SUM(ABS(it.qty)) as daily_consumption
         FROM inventory_txns it
         WHERE it.part_id = p.part_id
               AND it.type = 'OUT'
               AND it.created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
         GROUP BY DATE(it.created_at)
       ) daily_stats),
      1
    )
  )
```

```
        ELSE 999
    END as estimated_days_remaining
FROM parts p
WHERE p.qty < 20 -- 库存数量小于20的配件
ORDER BY estimated_days_remaining ASC;
```

4.3 配件出库操作

功能描述：工单使用配件时的库存扣减

```
-- 开始事务
START TRANSACTION;

-- 检查库存是否充足
SELECT qty FROM parts WHERE part_id = 5001 FOR UPDATE;

-- 如果库存充足，记录出库交易
INSERT INTO inventory_txns (part_id, order_id, qty, type, created_at)
VALUES (5001, 10001, -5, 'OUT', NOW());

-- 更新配件库存
UPDATE parts
SET qty = qty - 5
WHERE part_id = 5001;

-- 记录工单配件使用
INSERT INTO work_order_materials (order_id, part_id, qty, price)
VALUES (10001, 5001, 5, 25.00);

-- 提交事务
COMMIT;
```

5. 费用计算相关SQL

5.1 工单费用计算

功能描述：计算工单的人工费和配件费

```
-- 计算工单总费用
SELECT
    wo.order_id,
    -- 计算人工费
    COALESCE(
        SUM(wom.hours_worked * mp.hourly_rate),
        0
    ) as calculated_labor_fee,
    -- 计算配件费
    COALESCE(
```

```

        (SELECT SUM(wom2.qty * wom2.price)
         FROM work_order_materials wom2
         WHERE wom2.order_id = wo.order_id),
        0
    ) as calculated_material_fee,
    -- 计算总费用
    COALESCE(
        SUM(wom.hours_worked * mp.hourly_rate),
        0
    ) + COALESCE(
        (SELECT SUM(wom2.qty * wom2.price)
         FROM work_order_materials wom2
         WHERE wom2.order_id = wo.order_id),
        0
    ) as calculated_total_fee,
    -- 当前支付记录中的费用
    p.labor_fee as recorded_labor_fee,
    p.material_fee as recorded_material_fee,
    p.total_fee as recorded_total_fee
FROM work_orders wo
LEFT JOIN work_order_mechanics wom ON wo.order_id = wom.order_id
LEFT JOIN mechanic_profiles mp ON wom.mechanic_id = mp.mechanic_id
LEFT JOIN payments p ON wo.order_id = p.order_id
WHERE wo.order_id = 10001
GROUP BY wo.order_id, p.labor_fee, p.material_fee, p.total_fee;

```

5.2 更新支付记录

功能描述：根据实际工作时长和配件使用更新费用

```

-- 更新支付记录
UPDATE payments p
INNER JOIN (
    SELECT
        wo.order_id,
        COALESCE(SUM(wom.hours_worked * mp.hourly_rate), 0) as new_labor_fee,
        COALESCE(
            (SELECT SUM(wom2.qty * wom2.price)
             FROM work_order_materials wom2
             WHERE wom2.order_id = wo.order_id),
            0
        ) as new_material_fee
    FROM work_orders wo
    LEFT JOIN work_order_mechanics wom ON wo.order_id = wom.order_id
    LEFT JOIN mechanic_profiles mp ON wom.mechanic_id = mp.mechanic_id
    WHERE wo.order_id = 10001
    GROUP BY wo.order_id
) calc ON p.order_id = calc.order_id
SET
    p.labor_fee = calc.new_labor_fee,
    p.material_fee = calc.new_material_fee,

```

```
p.total_fee = calc.new_labor_fee + calc.new_material_fee
WHERE p.order_id = 10001;
```

6. 报表统计相关SQL

6.1 营业收入统计

功能描述：按时间段统计营业收入

```
-- 月度收入统计
SELECT
    DATE_FORMAT(p.paid_at, '%Y-%m') as month,
    COUNT(p.payment_id) as total_orders,
    SUM(p.labor_fee) as total_labor_revenue,
    SUM(p.material_fee) as total_material_revenue,
    SUM(p.total_fee) as total_revenue,
    AVG(p.total_fee) as avg_order_value
FROM payments p
WHERE p.paid_at IS NOT NULL
    AND p.paid_at >= DATE_SUB(NOW(), INTERVAL 12 MONTH)
GROUP BY DATE_FORMAT(p.paid_at, '%Y-%m')
ORDER BY month DESC;
```

6.2 技师工作量统计

功能描述：统计技师的工作效率和收入

```
-- 技师月度工作统计
SELECT
    u.name as mechanic_name,
    mp.trade,
    mp.hourly_rate,
    DATE_FORMAT(wo.created_at, '%Y-%m') as month,
    COUNT(DISTINCT wom.order_id) as completed_orders,
    SUM(wom.hours_worked) as total_hours,
    AVG(wom.hours_worked) as avg_hours_per_order,
    SUM(wom.hours_worked * mp.hourly_rate) as calculated_earnings
FROM users u
INNER JOIN mechanic_profiles mp ON u.user_id = mp.mechanic_id
INNER JOIN work_order_mechanics wom ON mp.mechanic_id = wom.mechanic_id
INNER JOIN work_orders wo ON wom.order_id = wo.order_id
WHERE wom.status = 'completed'
    AND wo.created_at >= DATE_SUB(NOW(), INTERVAL 6 MONTH)
GROUP BY u.user_id, u.name, mp.trade, mp.hourly_rate, DATE_FORMAT(wo.created_at, '%Y-%m')
ORDER BY month DESC, calculated_earnings DESC;
```


6.3 客户满意度统计

功能描述：统计客户反馈和满意度

```
-- 客户满意度统计
SELECT
    DATE_FORMAT(f.created_at, '%Y-%m') as month,
    COUNT(CASE WHEN f.type = 'rating' THEN 1 END) as total_ratings,
    AVG(CASE WHEN f.type = 'rating' THEN f.rating END) as avg_rating,
    COUNT(CASE WHEN f.type = 'rating' AND f.rating = 5 THEN 1 END) as
five_star_count,
    COUNT(CASE WHEN f.type = 'rating' AND f.rating = 4 THEN 1 END) as
four_star_count,
    COUNT(CASE WHEN f.type = 'rating' AND f.rating = 3 THEN 1 END) as
three_star_count,
    COUNT(CASE WHEN f.type = 'rating' AND f.rating = 2 THEN 1 END) as
two_star_count,
    COUNT(CASE WHEN f.type = 'rating' AND f.rating = 1 THEN 1 END) as
one_star_count,
    COUNT(CASE WHEN f.type = 'urge' THEN 1 END) as urge_count,
    -- 计算满意度百分比 (4星及以上)
    ROUND(
        COUNT(CASE WHEN f.type = 'rating' AND f.rating >= 4 THEN 1 END) * 100.0 /
        NULLIF(COUNT(CASE WHEN f.type = 'rating' THEN 1 END), 0),
        2
    ) as satisfaction_rate
FROM feedbacks f
WHERE f.created_at >= DATE_SUB(NOW(), INTERVAL 12 MONTH)
GROUP BY DATE_FORMAT(f.created_at, '%Y-%m')
ORDER BY month DESC;
```

7. 数据维护相关SQL

7.1 数据一致性检查

功能描述：检查系统数据的一致性

```
-- 检查配件库存与交易记录的一致性
SELECT
    p.part_id,
    p.name,
    p.qty as system_stock,
    COALESCE(SUM(it.qty), 0) as calculated_stock,
    (p.qty - COALESCE(SUM(it.qty), 0)) as difference
FROM parts p
LEFT JOIN inventory_txns it ON p.part_id = it.part_id
GROUP BY p.part_id, p.name, p.qty
HAVING difference != 0;
```

```
-- 检查支付记录与实际费用的一致性
SELECT
    p.order_id,
    p.labor_fee as recorded_labor,
    p.material_fee as recorded_material,
    p.total_fee as recorded_total,
    COALESCE(labor_calc.calculated_labor, 0) as calculated_labor,
    COALESCE(material_calc.calculated_material, 0) as calculated_material,
    (COALESCE(labor_calc.calculated_labor, 0) +
    COALESCE(material_calc.calculated_material, 0)) as calculated_total
FROM payments p
LEFT JOIN (
    SELECT
        wom.order_id,
        SUM(wom.hours_worked * mp.hourly_rate) as calculated_labor
    FROM work_order_mechanics wom
    INNER JOIN mechanic_profiles mp ON wom.mechanic_id = mp.mechanic_id
    GROUP BY wom.order_id
) labor_calc ON p.order_id = labor_calc.order_id
LEFT JOIN (
    SELECT
        order_id,
        SUM(qty * price) as calculated_material
    FROM work_order_materials
    GROUP BY order_id
) material_calc ON p.order_id = material_calc.order_id
WHERE
    ABS(p.labor_fee - COALESCE(labor_calc.calculated_labor, 0)) > 0.01
    OR ABS(p.material_fee - COALESCE(material_calc.calculated_material, 0)) > 0.01
    OR ABS(p.total_fee - (COALESCE(labor_calc.calculated_labor, 0) +
    COALESCE(material_calc.calculated_material, 0))) > 0.01;
```

7.2 数据清理

功能描述：清理过期或冗余数据

```
-- 清理超过2年的已完成工单反馈记录（保留评分记录）
DELETE FROM feedbacks
WHERE type = 'urge'
    AND created_at < DATE_SUB(NOW(), INTERVAL 2 YEAR);

-- 归档超过1年的库存交易记录到历史表
INSERT INTO inventory_txns_history
SELECT * FROM inventory_txns
WHERE created_at < DATE_SUB(NOW(), INTERVAL 1 YEAR);

-- 删除已归档的记录
DELETE FROM inventory_txns
WHERE created_at < DATE_SUB(NOW(), INTERVAL 1 YEAR);
```

8. 性能优化建议

8.1 索引优化

```
-- 创建复合索引优化查询性能
CREATE INDEX idx_work_orders_status_trade ON work_orders(status, trade);
CREATE INDEX idx_inventory_txns_part_created ON inventory_txns(part_id,
created_at);
CREATE INDEX idx_feedbacks_type_rating ON feedbacks(type, rating);
```

8.2 查询优化

```
-- 使用 EXPLAIN 分析查询执行计划
EXPLAIN SELECT /* 查询语句 */;

-- 定期更新表统计信息
ANALYZE TABLE users, work_orders, parts, inventory_txns;
```

8.3 定期维护

```
-- 优化表结构
OPTIMIZE TABLE work_orders;
OPTIMIZE TABLE inventory_txns;

-- 重建索引
ALTER TABLE work_orders DROP INDEX idx_work_orders_created_at, ADD INDEX
idx_work_orders_created_at(created_at);
```

9. 安全考虑

9.1 SQL注入防护

- 所有用户输入都通过参数化查询处理
- 使用Sequelize ORM的内置防护机制
- 定期审计和更新查询语句

9.2 权限控制

```
-- 创建只读用户用于报表查询
CREATE USER 'report_user'@'%' IDENTIFIED BY 'strong_password';
GRANT SELECT ON vehicle_repair_system.* TO 'report_user'@'%;

-- 创建备份用户
```

```
CREATE USER 'backup_user'@'localhost' IDENTIFIED BY 'backup_password';  
GRANT SELECT, LOCK TABLES ON vehicle_repair_system.* TO 'backup_user'@'localhost';
```